

Testing Phase Report

Backend, Frontend and Semantic Testing

Overview

The primary responsibility within the project was the execution of backend, frontend, and semantic testing activities for the Drone Routing Problem application. The main objective of these testing processes was to verify that the developed system satisfied both the functional requirements and the business logic defined during the software design phase. Particular attention was given to ensuring that the interaction between the user interface, backend services, and optimization algorithms remained stable and consistent under different operating conditions.

The project provides a platform for solving and comparing drone-assisted vehicle routing problems by utilizing two different optimization approaches: an exact solution based on the Gurobi optimizer and a heuristic solution based on a Genetic Algorithm. Since the application integrates multiple software components, a comprehensive testing process was required to validate the correctness and reliability of the overall workflow.

Backend Testing

During the backend testing phase, API endpoints were examined to ensure that user requests were correctly processed and that valid responses were generated. The communication between the frontend interface and the FastAPI backend was evaluated under different input conditions. One of the principal test scenarios involved selecting the Gurobi optimization method and submitting a routing problem instance to the server. The backend was expected to receive the dataset, execute the optimization model, and return the generated routes together with the objective value and execution statistics. The returned information was verified to ensure that the response structure matched the requirements of the frontend application.

A second backend scenario focused on the Genetic Algorithm module. The application was expected to produce a feasible routing solution without relying on the exact optimization engine. The generated routes and corresponding objective values were examined to verify that the heuristic process consistently produced valid outputs and that the execution was completed without unexpected failures.

The custom dataset upload mechanism was also tested extensively. Since users are allowed to provide their own routing instances, it was necessary to confirm that the backend correctly parsed and validated uploaded files. Several situations were evaluated, including valid datasets, incomplete datasets, and incorrectly formatted files. For invalid inputs, the system successfully rejected the request and generated appropriate error messages without interrupting the execution flow.

Frontend Testing

Frontend testing concentrated on verifying that all user interactions correctly activated their corresponding backend operations. Algorithm selection menus, file upload components, comparison options, and result visualization pages were examined through manual testing procedures. The displayed information was compared with the backend responses to ensure consistency throughout the application.

One representative frontend scenario involved selecting different optimization algorithms and confirming that the correct parameters were transmitted to the backend. Following the optimization process, the resulting routes, execution times, and objective values were expected to appear correctly within the graphical interface. Additional verification was performed for multilingual support and theme-switching functionality to ensure that these auxiliary features did not interfere with the core optimization workflow.

Semantic Testing

Semantic testing represented a significant part of the quality assurance process. Unlike conventional functional testing, semantic validation focuses on determining whether the generated outputs are logically correct according to the constraints of the problem domain. For the Drone Routing Problem application, successful execution alone was not considered sufficient; the produced routing solutions also had to satisfy the mathematical and operational requirements of the model.

One semantic validation scenario involved confirming that every customer location was visited exactly once. The generated routes obtained from both the Gurobi solver and the Genetic Algorithm were inspected to ensure that no customer was omitted and that no location appeared multiple times. This verification was essential because duplicate or missing visits would invalidate the generated solution regardless of the successful execution of the software.

Another semantic test focused on drone operational constraints. The resulting routes were analyzed to verify that drone activities respected their operational limitations throughout the mission. The generated solutions were expected to ensure that drone tasks remained within acceptable constraints and that drones returned to the truck after completing their assigned deliveries. Multiple datasets were used during this process to evaluate the consistency of the implementation.

Comparative semantic validation was also performed between the exact optimization approach and the heuristic approach. Although the two methods naturally produced different execution times and objective values, both solutions were required to satisfy the same routing constraints and generate logically valid delivery plans. The comparison module was examined to verify that the application correctly presented the performance metrics of both algorithms without introducing inconsistencies.

Edge Case & Exceptional Input Handling

Throughout the testing process, both normal and exceptional use cases were evaluated. Invalid inputs, missing files, unsupported configurations, and incorrect parameter combinations were intentionally introduced to observe the behavior of the application under adverse conditions. In each tested scenario, the system successfully handled the interaction and generated appropriate responses.

Conclusion

As a result of these testing activities, the backend services, frontend interface, and optimization modules were verified to be properly integrated and consistent with the project requirements. The overall quality assurance process confirmed that the application was capable of generating valid routing solutions, processing user-defined datasets, and accurately presenting optimization results through the graphical user interface.

The testing activities included the preparation and execution of test scenarios, the verification of backend and frontend functionality, the semantic validation of routing solutions, and the confirmation that the software product satisfied both technical and logical requirements prior to project completion.
